Technical Digest: Reversing the Sonic Delta Origins Bank-Switching Mapper

Executive Summary

Sonic Delta Origins (historically known as *Sonic Delta 40Mb*) is a massive Sega Genesis/Mega Drive ROM hack created by Esrael Neto. It natively stitches *Sonic 1, 2, 3*, and *Sonic & Knuckles* into a single continuous game running on the *Sonic 3 & Knuckles* engine framework.

The developer asserted that standard automated disassembly would fail due to thousands of "incorrect pointers" caused by custom memory mapping. This digest provides the exact technical strategy, architectural mapping, and automated Python scripts required to bypass these roadblocks, map out the architecture inside Ghidra, and achieve a clean, workable source code disassembly.

Hardware Architecture & Memory Layout

The Motorola 68000 CPU inside the Sega Genesis has a native direct addressing limit of 32 Megabits (4 Megabytes) for ROM space (ranging from 0x000000 to 0x3FFFFF). Since the unified game architecture of Sonic Delta Origins expands the file size to 40 Megabits (5 Megabytes), it utilizes hardware bank-switching. Neto repurposed the official commercial memory mapping logic created by SEGA/Capcom for the 1993 release of **Super Street Fighter II (SSFII)**.

Memory Window	Slot Name	Hardware Register	Behavior / Purpose
0x000000-0x07FFFF	Bank 00 (Fixed)	*Locked*	Main engine vectors, boot routines, menu selectors.
0x080000-0x0FFFFF	Slot 1	0xA130F1	Dynamically swaps active game code blocks.
0x100000-0x17FFFF	Slot 2	0xA130F3	Dynamically swaps active game code blocks.
0x180000-0x1FFFFF	Slot 3	0xA130F5	Commonly used for music driver/DAC asset swaps.
0x200000-0x27FFFF	Slot 4	0xA130F7	Dynamically swaps active level/art data blocks.
0x280000-0x2FFFFF	Slot 5	0xA130F9	Dynamically swaps active level/art data blocks.
0x300000-0x37FFFF	Slot 6	0xA130FB	Dynamically swaps active level/art data blocks.

0x380000-0x3FFFFF	Slot 7	0xA130FD	Dynamically swaps active level/art data blocks.

Ghidra Workspace Configuration Blueprint

To cleanly resolve memory references, you must map the binary into a custom virtual space in Ghidra that mimics the physical layout of the SSFII cartridge hardware configuration:

1. **Import Base Block:** Import the extracted bank_00.bin file as a flat binary program. Select Language: 68000 (M68000:BE:32:default).

2. **Establish Hardware Slots:** Open Window -> Memory Map. Proactively define 7 uninitialized memory blocks corresponding to the address ranges of Slot 1 through Slot 7 listed above. Ensure the **Execute** attribute flag is checked, as the processor branches into these regions.

3. **Map I/O Registers:** Create an uninitialized block named GENESIS_HARDWARE_REGS from 0xA00000 with a length of 0x200000. This tracks and flags writes to the bank selectors.

4. **Leverage Overlay Blocks:** When tracking indicates a specific segment is active (e.g., Bank 08 is loaded into Slot 4), use File -> Add to Program... to load that bank's binary chunk. Set Block Type to **Overlay** and explicitly anchor its Base Address to 0x200000. Ghidra allows you to toggle overlays on the fly to resolve pointer mutations.

Automation Toolset Scripts

Script 1: External File-System ROM Splitter (Python)

```
import os
def split_sonic_delta_rom(rom_path, output_dir="extracted_banks"):
    BANK_SIZE = 0x80000
    if not os.path.exists(rom_path): return
    if not os.path.exists(output_dir): os.makedirs(output_dir)
    with open(rom_path, "rb") as rom_file:
        bank_number = 0
        while True:
            bank_data = rom_file.read(BANK_SIZE)
            if not bank_data: break
            start_offset = bank_number * BANK_SIZE
            end_offset = start_offset + len(bank_data) - 1
            out_name = f"bank_{bank_number:02d}_[0x{start_offset:06X}-0x{end_offset:06X}].bin"
            with open(os.path.join(output_dir, out_name), "wb") as out_file:
                out_file.write(bank_data)
            bank_number += 1
split_sonic_delta_rom("Sonic_Delta_Origins.md")
```

Script 2: Ghidra Static Hardware Mapper Tracker (Python)

```
from ghidra.program.model.symbol import SourceType
MAPPER_REGISTERS = {
    0xA130F1: "SSF2_BANK_SLOT1", 0xA130F3: "SSF2_BANK_SLOT2",
    0xA130F5: "SSF2_BANK_SLOT3", 0xA130F7: "SSF2_BANK_SLOT4"
}
def analyze_mapper_writes():
    listing = getCurrentProgram().getListing()
    instruction = listing.getInstructions(toAddr(0x000000), True)
    while instruction.hasNext() and not currentMonitor.isCancelled():
```

```
        ins = instruction.next()
        if "move" in ins.getMnemonicString().lower():
            for i in range(ins.getNumOperands()):
                op_addr = ins.getAddress(i)
                if op_addr and op_addr.getOffset() in MAPPER_REGISTERS:
                    reg_name = MAPPER_REGISTERS[op_addr.getOffset()]
                    bank_val = ins.getDefaultOperandRepresentation(0)
                    ins.setComment(ins.PLATE_COMMENT, f"BANK SWITCH: {reg_name} = {bank_val}")
analyze_mapper_writes()
```

Script 3: Ghidra Engine Subsystem Signature Scanner (Python)

```
from ghidra.program.model.symbol import SourceType
from java.lang import Byte
SIGNATURES = {
    "Decomp_Kosinski": "48 e7 fe fe 20 50 22 51 43 f9 00 00 00 00 42 80 10 18",
    "Init_SoundDriver": "30 3c 01 00 13 fc 01 00 00 a1 11 00 13 fc 01 00 00 a1 12 00"
}
def scan():
    mem = getCurrentProgram().getMemory()
    st = getCurrentProgram().getSymbolTable()
    for name, p_str in SIGNATURES.items():
        p_bytes = [int(b, 16) for b in p_str.split(" ")]
        j_bytes = [Byte(b).byteValue() for b in p_bytes]
        j_mask = [Byte(0xFF).byteValue() for _ in p_bytes]
        addr = mem.findBytes(toAddr(0x000000), j_bytes, j_mask, True, currentMonitor)
        if addr and not st.getPrimarySymbol(addr):
            st.createLabel(addr, name, SourceType.USER_DEFINED)
scan()
```